arXiv:2007.00017v2 [quant-ph] 6 Dec 2021

# Dynamic Portfolio Optimization with Real Datasets Using Quantum Processors and Quantum-Inspired Tensor Networks

Samuel Mugel,[1] Carlos Kuchkovsky,[2] Escolástico Sánchez,[2] Samuel Fernández-Lorenzo,[2] Jorge Luis-Hita,[2] Enrique Lizaso,[3] and Román Orús[3,4,5]

[1] *Multiverse Computing, Banting Institute, 100 College Street, ONRamp Suite 150, Toronto, ON M5G 1L5 Canada*
[2] *BBVA Research & Patents, Calle Sauceda 28, 28050 Madrid, Spain*
[3] *Multiverse Computing, Paseo de Miramón 170, E-20014 San Sebastián, Spain*
[4] *Donostia International Physics Center, Paseo Manuel de Lardizabal 4, E-20018 San Sebastián, Spain*
[5] *Ikerbasque Foundation for Science, Maria Diaz de Haro 3, E-48013 Bilbao, Spain*

In this paper we tackle the problem of dynamic portfolio optimization, i.e., determining the optimal trading trajectory for an investment portfolio of assets over a period of time, taking into account transaction costs and other possible constraints. This problem is central to quantitative finance. After a detailed introduction to the problem, we implement a number of quantum and quantum-inspired algorithms on different hardware platforms to solve its discrete formulation using real data from daily prices over 8 years of 52 assets, and do a detailed comparison of the obtained Sharpe ratios, profits and computing times. In particular, we implement classical solvers (Gekko, exhaustive), D-Wave Hybrid quantum annealing, two different approaches based on Variational Quantum Eigensolvers on IBM-Q (one of them brand-new and tailored to the problem), and for the first time in this context also a quantum-inspired optimizer based on Tensor Networks. In order to fit the data into each specific hardware platform, we also consider doing a preprocessing based on clustering of assets. From our comparison, we conclude that D-Wave Hybrid and Tensor Networks are able to handle the largest systems, where we do calculations up to 1272 fully-connected qubits for demonstrative purposes. Finally, we also discuss how to mathematically implement other possible real-life constraints, as well as several ideas to further improve the performance of the studied methods.

## I. INTRODUCTION

In quantitative finance, portfolio optimization is the problem of selecting the best distribution of assets that optimizes some objective function [1]. Typically, this objective function tries to maximize the expected returns and minimize the financial risk. The problem gets more complicated if we do it dynamically, i.e., optimize the investment portfolio over a series of consecutive trading days. In this *dynamic portfolio optimization*, the goal is to determine the optimal trading trajectory over the considered period of time, i.e., the optimal decisions that should be taken (or should have been taken) by a broker in order to maximize the overall return at the end of the time period. The dynamic problem is more complex because transactions' costs and transactions' market impact must be taken into account, as well as other possible constraints. In practice, it is well-known that this is an *intractable* problem for generic instances.

Parallel to the above, it has been understood recently that quantum and quantum-inspired computing can help in solving hard financial problems [2, 3]. For instance, a quantum computer should be able to solve more efficiently and with more accuracy problems related to pricing of financial derivatives [4–6], prediction of financial crashes [7, 8], detection of arbitrage cycles [9], credit scoring [10], and identification of several types of fraud, among many applications. Portfolio optimization is no exception, as observed in several preexisting studies [11, 12]. Yet, to the best of our knowledge, none of these works aimed to solve the problem on real datasets, and no comparison has ever been done openly and democratically between different methods and hardware platforms.

In this paper we implement several quantum and quantum-inspired algorithms for dynamic portfolio optimization, and run them for the first time (as fas as we are aware) with real data corresponding to daily prices over 8 years of 52 assets. In particular, we implement D-Wave Hybrid quantum annealing, a Variational Quantum Eigensolver (VQE) on a quantum processor of IBM-Q, a new VQE-inspired algorithm which we call "VQE Constrained" also on IBM-Q, and a quantum-inspired Tensor Network (TN) optimization algorithm (which is also the first implementation of a TN algorithm to solve a real and practical financial problem). We benchmark our algorithms using two classical methods: a Gekko solver (a Python-based optimization suite [13]) and an exhaustive solver. We expose how preprocessing is performed, and reduce the problem's dimensionality using a clustering algorithm. We then do a detailed comparison of all the results, focusing on the obtained Sharpe ratios and computing times. From our comparison we conclude that, as of today, D-Wave Hybrid and Tensor Networks are able to handle the largest systems, where we do calculations up to 1272 fully-connected qubits for demonstrative purposes. Interestingly, we see that there is no clear answer as to which is the "best" algorithm and/or hardware platform to deal with large systems, as this depends strongly on different figures of merit.

This paper is organized as follows. In Sec. II we give

an overview of the dynamic portfolio optimization problem. We show how it can be expressed Quadratic Unconstrained Binary Optimization (QUBO) problem, and discuss the differences between the continuous and discrete formulations. In Sec. III we give a very brief overview of the D-Wave hybrid, VQE, and TN algorithms. In Sec. IV we expose the data-preparation procedure, which reduces the problem's dimensionality by identifying irrelevant assets and performing clusterization. Sec. V compares results obtained using the Gekko, Exhaustive, D-Wave Hybrid, VQE, VQE-Constrained, and TN solvers. Sec. VI discusses industry relevant next steps, such as the inclusion of more real-life constraints (e..g, market impact, exact linear transaction costs, and the so-called $10-5-40$ rule) and potential performance improvements. Finally, in Sec. VII we wrap up and conclude.

## II. PROBLEM OVERVIEW

The dynamic portfolio optimization problem can be expressed in a form amenable to a quantum computer. In what follows, we present the problem in some technical detail. Where possible, we use the notation of [11].

### A. Optimal dynamic portfolio

#### 1. Generalities

In the dynamic version of the so-called *Modern Portfolio Theory* (or *Mean Variance Analysis*), we deal with the issue of allocating weigths to a number of assets over a period of time, in order to maximize the overall return at the end of the period. More specifically, for $N$ assets we consider an $N$-dimensional vector of weigths $\overline{\omega}_t$. Each of its component $\overline{\omega}_{n,t}$ is the weigth of asset $n$ at time $t = t_i, t_i + 1, \ldots, t_f$, where $t_i$ and $t_f$ are respectively the initial and final trading (rebalancing) times, being the number of trading steps $N_t = t_f - t_i + 1$. We also define $\mu_t$, assets' forecast returns at time $t$, and $\Sigma_t$, the assets' covariance at time $t$. $\mu_t$ is a $N$-length vector while $\Sigma_t$ is and $N \times N$ matrix. For a given trading trajectory (i.e., a given set of vectors $\{\overline{\omega}_{t_i}, \ldots, \overline{\omega}_{t_f}\}$), the overall return is given by

$$\text{Return} \equiv \sum_{t=t_i}^{t_f} \mu_t^T \overline{\omega}_t, \tag{1}$$

and the risk of the trajectory is defined as

$$\text{Risk} \equiv \frac{1}{2} \sum_{t=t_i}^{t_f} \overline{\omega}_t^T \Sigma_t \overline{\omega}_t. \tag{2}$$

Notice that $\overline{\omega}_t^T \Sigma_t \overline{\omega}_t$ is the variance of the portfolio return at time $t$. In practical situations, one typically measures risk at time $t$ in terms of the *volatility*, which is the square-root of this variance. Nevertheless, Eq.(2) is a convenient way to quantify the risk in the dynamic setting since it makes the optimization problem better behaved, being the prefactor $1/2$ a convention.

The goal of modern portfolio theory is to find the trajectory which maximizes returns for a fixed risk. It is common to request the total investment at any given time is fixed, i.e.,

$$\sum_{n=1}^{N} \overline{\omega}_{n,t} = K \quad \forall t, \tag{3}$$

with $K$ the total investment [14]. Let us define at this step the normalized weigths

$$\omega_{n,t} \equiv \frac{\overline{\omega}_{n,t}}{K}, \tag{4}$$

so that their sum at every time $t$ is equal to one. In terms of these normalized weigths, the problem can be solved by finding the trajectory $\{\omega_{t_i}, \ldots, \omega_{t_f}\}$ which minimizes:

$$\mathcal{H} = \sum_{t=t_i}^{t_f} -\mu_t^T \omega_t + \frac{\gamma}{2} \omega_t^T \Sigma_t \omega_t + \rho \left(u^T \omega_t - 1\right)^2. \tag{5}$$

By analogy to quantum mechanics, we shall refer to the above cost function $\mathcal{H}$ as *Hamiltonian*. $\gamma$ is the *risk aversion*, which tunes the eagerness of the investor to explore risky trajectories, and $\rho$ is a Lagrange multiplier that imposes the constraint in Eq. (3) as a penalty term. We have introduced the $N$-dimensional vector $u$, with $u_n = 1\ \forall n$, which makes the constraint in Eq. (3) more compact. The Lagrange multiplier $\rho$ is fine-tuned in order to satisfy Eq. (3).

Note that Eq. (5) can also be written as

$$\mathcal{H} = \sum_{t=t_i}^{t_f} h_t, \tag{6}$$

with

$$h_t \equiv -\mu_t^T \omega_t + \frac{\gamma}{2} \omega_t^T \Sigma_t \omega_t + \rho \left(u^T \omega_t - 1\right)^2. \tag{7}$$

Thus, the Hamiltonian is *diagonal* in time. This implies that the optimal trading trajectory is simply *the concatenation of the optimal portfolios at each time* $t$. As we will see in the following, if this is the case, then the problem can be solved analytically (in the continuous variable limit). This is not the case when the objective function has terms correlating different times with each other.

Note that the $\omega_{n,t}$ are interpreted as a *percentage* of the total investment. For instance, having $\omega_{n,t} = 0.1$ means that we invest a 10% of our total amount $K$ in asset $n$ at time $t$. Additionally, one may also introduce a cap $K'$ on the maximum amount that can be invested for each asset, i.e., $\omega_{n,t} \leq K'/K$.

We measure the quality of a portfolio by the so-called *Sharpe ratio*.

$$\text{Sharpe} \equiv \frac{\sum_{t=t_i}^{t_f} \mu_t^T \omega_t}{\sqrt{\sum_{t=t_i}^{t_f} \omega_t^T \Sigma_t \omega_t}}, \tag{8}$$

This quantifies the amount of return per unit of risk of trading trajectory. Notice that the numerator is the Return, and not the Profit, which would imply removing all possible additional costs (such as transaction costs, to be discussed later). Notice also that for e.g. one rebalancing step, the denominator is the normalized volatility, understood as the square root of the variance of the normalized returns. A large Sharpe ratio means a large return for the risk that is assumed, whereas a ratio close to zero means the opposite, and a negative ratio means losses instead of profits.

#### 2. Transaction costs

The problem stated above is simplistic as it does not account for transaction costs. These frequently are comparable to the profit incurred by a given portfolio. The transaction costs are typically a percentage of the transaction (whether buying or selling assets). They can be expressed in terms of unnormalized weigths as:

$$\text{Cost} \equiv \sum_{t=t_i}^{t_f} \nu_t |\Delta \overline{\omega}_t| = \sum_{t=t_i}^{t_f} \sum_{n=1}^{N} \nu_{nt} |\overline{\omega}_{n,t+1} - \overline{\omega}_{n,t}|, \tag{9}$$

with $\nu_{nt}$ the cost percentage (e.g., $\nu_{nt} = 0.001$ for 10 Basis Points (BPS), meaning a cost of the 0.1% of the total amount of the transaction). The objective function which accounts for these costs in terms of normalized weigths is given by

$$\mathcal{H} = \sum_{t=t_i}^{t_f} -\mu_t^T \omega_t + \frac{\gamma}{2} \omega_t^T \Sigma_t \omega_t + \nu_t |\Delta \omega_t| + \rho \left(u^T \omega_t - 1\right)^2. \tag{10}$$

Note that this is not of the form in Eq. (6), since $\Delta\omega_t$ correlates times $t$ and $t+1$.

The (percentual) transaction costs are not polynomial in the variables $\omega_{n,t}$ because of the absolute value function. This could limit the applicability of some quantum optimization methods. To get around this problem, we can Taylor-expand the absolute value, in a way similar to the expansion in Ref.[7] for a step-function. Alternatively, we could introduce ancillary qubits to treat this problem exactly, as explained in Sec. VI. Here, we choose to approximate Eq. (9) by a parabola in the considered range of $\omega_{n,t}$:

$$\nu_t |\Delta \omega_t| \approx \Delta \omega_t^T \Lambda_t \Delta \omega_t, \quad \omega_{n,t} \in \left[0, \frac{K'}{K}\right], \tag{11}$$

with $\Lambda_t$ the best *matrix* of transaction costs at time $t$ that is compatible and realistic with market conditions. This is an excellent approximation to Eq. (9) when the $\omega_{n,t}$ are discrete variables and $K'/K$ is small. The cost function therefore reduces to:

$$\mathcal{H} = \sum_{t=t_i}^{t_f} -\mu_t^T \omega_t + \frac{\gamma}{2} \omega_t^T \Sigma_t \omega_t + \lambda (\Delta \omega_t)^2 + \rho \left(u^T \omega_t - 1\right)^2, \tag{12}$$

with $\lambda$ the optimal parabolic coefficient for the transaction costs, as discussed above. Finally, let us remark that in this setting, the percentual profits of the trading trajectory are given by the expression

$$\text{Profit} \equiv \sum_{t=t_i}^{t_f} \left(\mu_t^T \omega_t - \lambda (\Delta \omega_t)^2\right), \tag{13}$$

i.e., the percentual returns minus the percentual costs.

### B. Continuous versus discrete formulations

In this article, we chose to discuss the portfolio optimization problem with discrete variables, which is more relevant to big industry players, as investment funds typically trade in large, discrete amounts. In this section, we will briefly discuss the case where the asset allocations $\omega_{n,t}$ can be approximated by continuous variables. This problem is comparatively simpler, as it gives us access to the full toolbox of differential calculus.

#### 1. Continuous asset allocations

When no transaction costs are present, the problem can actually be solved *exactly*. The minimization of Eq. (6) can then be written as

$$\frac{\partial \mathcal{H}}{\partial \omega_t^T} = \frac{\partial h_t}{\partial \omega_t^T} = 0 \quad \forall t \tag{14}$$

which, using $\mu_t^T \omega_t = (\mu_t^T \omega_t + \omega_t^T \mu_t)/2$ [15], amounts to

$$-\frac{1}{2}\mu_t - \rho u + \left(\frac{\gamma}{2}\Sigma_t + \rho u \cdot u^T\right) \omega_t = 0. \tag{15}$$

Therefore, the optimal solution at time $t$ is given by

$$\omega_t = \left(\frac{\gamma}{2}\Sigma_t + \rho u \cdot u^T\right)^{-1} \left(\frac{1}{2}\mu_t + \rho u\right), \tag{16}$$

and the optimal dynamic portfolio is just the concatenation of the optimal portfolios at each time $t$. For completeness, one could also get an equation for the multiplier $\rho$:

$$\frac{\partial \mathcal{H}}{\partial \rho} = \frac{\partial h_t}{\partial \rho} = 0 \quad \forall t, \tag{17}$$

which in the end simply implies:

$$u^T \omega_t = 1. \tag{18}$$

This is nothing but the condition in Eq. (3), implying:

$$u^T \left(\frac{\gamma}{2}\Sigma_t + \rho u \cdot u^T\right)^{-1} \left(\frac{1}{2}\mu_t + \rho u\right) = 1, \tag{19}$$

which is indeed an equation for $\rho$ at each time $t$. In practice, though, it is much easier to simply use Eq. (16) for a sufficiently large $\rho$, and check a posteriori that Eq. (3) is satisfied up to some degree of accuracy.

When transaction costs are included, it is no longer possible to solve the problem analytically. In the continuous-time limit, we can recast the problem as a set of nonlinearly coupled ordinary differential equations. To do so, we notice that in the discrete formulation the increment in time $\Delta t$ between two consecutive time steps is $\Delta t = 1$. The continuous-time limit can then be taken as $\Delta t \to 0$, implying

$$\sum_{t=t_i}^{t_f} \Delta t \to \int_{t_i}^{t_f} dt, \qquad \frac{\Delta \omega_t}{\Delta t} \to \dot{\omega}, \tag{20}$$

with $\dot{\omega}$ the *time derivative* of the vector of asset allocations $\omega$, which is now a vector field. In this limit, the cost function is given by

$$\mathcal{S} = \int_{t_i}^{t_f} L(\omega, \dot{\omega}, t)\ dt. \tag{21}$$

We will refer to $\mathcal{S}$ as the *action* and $L(\omega, \dot{\omega}, t)$ as the *Lagrangian*, by analogy to physics. The Lagrangian is obtained by taking the continuous-time limit of the cost function $\mathcal{H}$, Eq. (12). Finding the optimal trading trajectory thus reduces to a conventional functional minimization problem. Our goal is to find the time-path of $\omega$ which minimizes the action $\mathcal{S}$:

$$\frac{\delta \mathcal{S}}{\delta \omega} = 0. \tag{22}$$

As is well-known, the solution to this equation corresponds to the Euler-Lagrange equations for the Lagrangian, i.e.,

$$\frac{\partial L}{\partial \omega} - \frac{d}{dt}\frac{\partial L}{\partial \dot{\omega}} = 0. \tag{23}$$

For a specific problem at hand, it is easy to write the Lagrangian and unfold the above equation, resulting in a set of nonlinearly coupled ordinary differential equations for the asset allocations $\omega_n(t)$ with initial conditions at $t = t_i$. In this limit, one can thus solve the dynamic portfolio optimization problem using the wide variety of algorithms for systems of coupled differential equations. As an example, for Eq.(12) the Lagrangian is given by

$$L(\omega, \dot{\omega}, t) = -\mu^T \omega + \frac{\gamma}{2}\omega^T \Sigma\ \omega + \lambda(\dot{\omega})^2 + \rho\left(u^T\omega - 1\right)^2, \tag{24}$$

with $\omega, \dot{\omega}, \mu$ and $\Sigma$ being time-dependent.

### 2. Discrete asset allocations

In industry, the rebalancing is done at discrete time steps $t$, and it is common for funds to trade assets in large, discrete packages.

In this setting, the problem is naturally recast as a Quadratic Unconstrained Binary Optimization (QUBO). For this, we choose a binary encoding of each variable $\omega_{n,t}$ in terms of $N_q$ bits $x_{n,t,q}$. There are several options for this encoding, as discussed, e.g., in Ref.[11]. For simplicity in this paper we choose to work with the binary encoding

$$\omega_{n,t} = \frac{1}{K}\sum_{q=0}^{N_q-1} 2^q x_{n,t,q}, \tag{25}$$

where $x_{n,t,q} = 0, 1$. By construction, we have that the maximum investment per asset is $K' = 2^{N_q} - 1$, which is naturally included in the formalism. Investments go also in discrete packages of amount 1. Substituting Eq. (25) into, e.g., (12) results in a QUBO problem for the $N_{tot} = N \times N_t \times N_q$ bit variables, i.e., finding the optimal portfolio weigths at any given time is therefore equivalent to finding the ground state (i.e. the minimum over the variables $\{x_{n,t,q}\}$) of the classical Hamiltonian

$$\mathcal{H} = x^T Q x \tag{26}$$

with $x \in \{0,1\}^{N_{tot}}$ the bit-vector and $Q \in \mathbb{R}^{N_{tot}\times N_{tot}}$ the corresponding QUBO matrix, which can be easily derived from Eqs.(12,25). To solve this problem on a quantum computer, we quantize Eq. (26) by promoting the bit variables $\{x_{n,t,q}\}$ to qubit operators $\{\hat{x}_{n,t,q}\}$ with eigenstates $|0\rangle$ and $|1\rangle$. Our conclusion at this step is that the portfolio optimization problem, with discrete investments – as happen in real life –, is a *natural* problem for quantum and quantum-inspired methods. Its formulation is directly a QUBO, by construction, without the need of further mapping the problem to anything else: we can feed this problem to a quantum or quantum-inspired solver *as is*.

### 3. Discrete problem complexity

Note that, by applying the transformation $x_i = (1 + s_i)/2$, Eq. (26) can be mapped to finding the ground state of an Ising spin glass:

$$\mathcal{H} = \sum_{i,j=0}^{N_{tot}-1} J_{ij} s_i s_j, \tag{27}$$

where the $s_i = \pm 1$ are spin variables. The couplings $J_{ij}$ can be derived from Eq. (26).

Ising spin-glasses are known to be NP-Hard in the generic case [16], demonstrating that the portfolio optimization problem can be computationally costly. This

is true even when there are no transaction costs. The optimal trading trajectory is then the concatenation of optimal portfolios at each time step $t$. Finding the optimal trajectory there means solving $N_t$ independent optimization problems for $N \times N_q$ bits each. The instantaneous problem is itself an Ising spin glass which, in the worst case, may correspond to a hard instance (very much unlike in the continuous formulation, which is exactly solvable!).

## III. METHODS OVERVIEW

In this work we use a variety of methods and hardware implementations to solve the dynamic portfolio optimization problem in its discrete formulation. These are the following:

1. Classical: Gekko solver, exhaustive solver.
2. Quantum annealing: D-Wave Hybrid.
3. Quantum universal: VQE, VQE Constrained.
4. Quantum inspired: TN solver.

The classical methods were implemented as a benchmark of the rest of the algorithms. Gekko is a library which offers tools for non-convex, integer optimization problems [13]. The exhaustive solver is a brute-force search over valid configurations of the minimum of the cost function. Let us now make a brief overview of the rest of the methods.

### A. D-Wave Hybrid

As is well-known, quantum annealing is a type of quantum algorithm based on the ideas of adiabatic quantum computation [17], and is particularly well-suited to solve optimization problems. This process is similar to classical or simulated annealing, where thermal fluctuations allow the system to jump between different local minima in the energy landscape. In quantum annealing, the jumps are mainly driven by quantum tunneling events, which allow for a more efficient exploration of the landscape of local minima, especially when the energy barriers are tall and narrow.

In this work we used the quantum annealer provided by D-Wave, in particular the so-called D-Wave 2000Q processor, which gives access to 2048 non-coherent qubits coupled through the so-called chimera graph. This architecture allows us to solve problems for up to 65 fully-connected qubits, due to embedding overheads. The D-Wave Hybrid algorithm uses a hybrid classical-quantum strategy to overcomes this limitation, allowing us to deal with much larger problems. In a nutshell, D-Wave Hybrid breaks down problems which are larger than the capability of the quantum processor into parts. These are subsequently recombined to produce the solution.

### B. Variational Quantum Eigensolver

The Variational Quantum Eigensolver (VQE) [18] is a hybrid quantum-classical algorithm for optimization. The idea is to do a variational optimization of a quantum state in order to obtain an approximation to the ground state of a Hamiltonian. This idea is quite generic, but the point of the VQE algorithm is that the ansatz quantum state is, itself, a real quantum state that is implemented on a quantum processor by some quantum circuit. The gates in the circuit depend on parameters, which are the variational parameters of the algorithm. After estimating the energy of the quantum state via sampling, the parameters are then fine-tuned to lower the energy (using, e.g., conjugate gradient). After a number of iterations, the energy converges, producing an approximation to the desired ground state. The performance of VQE depends strongly on several aspects, but most importantly on the choice of variational quantum circuit. In some cases, the search for some complex ground states require of a strongly-entangling quantum circuit, in turn increasing the complexity of the algorithm. However, VQE is still a good option as an optimization tool in current Noisy Intermediate-Scale Quantum (NISQ) processors [19].

For the sake of this paper, we implemented VQE optimization in the quantum processors of IBM-Q, up to 12 qubits. The circuit ansatz for VQE optimization is shown in Fig. 1. It is based on a strategy of strongly entangling layers, inspired by the circuit-centric classifier design from Ref.[20], with single-qubit rotations around the $y$-axis. In particular, we developed our own VQE algorithm using Xanadu's *PennyLane* library for quantum machine learning [21], which is well-suited to run on IBM's quantum backend. Our variational quantum circuit consisted of 82 C-NOTs and 24 variational one-qubit rotations, as shown in Fig. 1.

Moreover, in order to tackle slightly-larger problems than those reachable by plain-vanilla VQE, we implemented a *new and original* approach which we call *VQE-Constrained* for dynamic portfolio optimization. The idea here is quite simple: use VQE to sample several low-energy states at every rebalancing time $t$, and then use a classical approach to find which combination of these provides the highest returns over the whole trading time period. This approach is inspired by low-energy-subspace methods in physics. It follows the intuition that the optimal portfolio can be built in most cases from a combination of near-optimal states. Thus, we identify these states using VQE (this is the computationally expensive part), and then recombine them and estimate their associated profits classically (which is computationally cheap). Our strategy here was to look for the best 10 solutions at each trading step using VQE. Then, in the post-processing, we tried all possible combinations until finding the one that mininized the cost function.

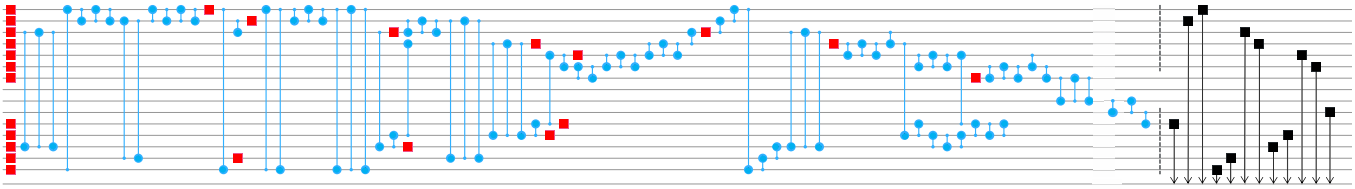

FIG. 1. (color online) Quantum circuit used for VQE and VQE-Constrained algorithms. The system had 15 qubits on the whole, of which 12 were used in the ansatz. The initial state was $|0\rangle^{\otimes 12}$. In the figure, single-site boxes are one-qubit $y$-rotations, and the two-qubit gates are CNOTs (target qubit being the large circle). In the end, measurements in the computational basis (boxes after the vertical dotted line) are performed on each qubit and stored in a classical register (as shown by the arrows).

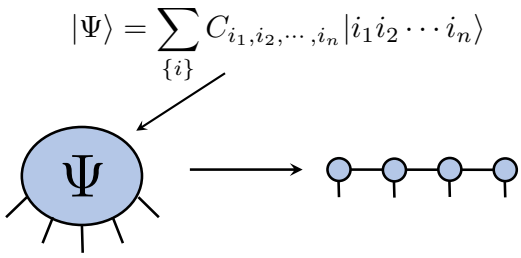


FIG. 2. The coefficient of the quantum state of $n$ qubits is a tensor with exponentially many coefficients in the system's size. The inner structure of this tensor is that of a tensor network, which is a network of tensors interconnected by ancillary indices that take into account the structure and amount of entanglement in the quantum state. We represent this here using diagrams, where shapes correspond to tensors, lines to indices, and lines connecting shapes to contracted (summed) common indices. The tensor network on the right hand side is an example of Matrix Product State. "Open" – uncontracted – lines in the tensor network correspond to the degrees of freedom of the original physical qubit (for the case at hand, one line per qubit).

### C. Tensor Networks

TNs are representations of complex quantum states based on their local entanglement structure [22, 23]. Take for instance a system of $n$ qubits. Any wave-function of the system can be described inefficiently just by giving its $O(2^n)$ coefficients in the computational basis. As such, these coefficients can be understood as a tensor with $n$ indices, where each index takes two possible values (say, 0 and 1). We could then think of replacing this huge, nasty tensor, by a network of interconnected tensors with less coefficients, see Fig. 2 for an example. This construction defines a TN, with each subsystem in the figure corresponding in practice to the Hilbert space of a qubit. Constructed in this way, the TN depends on $O(\text{poly}(n))$ parameters only, assuming that the rank of the interconnecting indices is upper-bounded by a parameter $D$, which is called "bond dimension". Similarly, interconnecting indices in the network are also called "bond indices", and provide the structure of the many-body entanglement in the quantum state. Any $D > 1$ provides an entangled quantum state.

As is well-known in physics, TNs are a natural tool to solve optimization problems. People have been using them as an ansatz to approximate low-energy eigenstates of Hamiltonians, and many algorithms have been invented to this aim (see e.g. Ref.[23] and references therein). The idea here is that, by mapping optimization problems to Hamiltonian eigenvalue problems, as done in quantum annealing, we can then use the huge machinery of TN techniques and algorithms to solve the optimization problem at hand.

In our case, we implemented an optimization strategy over the so-called Matrix Product States (MPS) [24]. This family of states has been tested already in a variety of algorithms for many physical examples. For practical reasons, here we use imaginary-time evolution as optimization method. Moreover, in order to improve the performance, we also tailored the actual implementation of our optimization to the specifics of our problem.

## IV. DATA PREPARATION

As a first step, we benchmarked our different algorithms for the optimization problem using random data. Studying real data proved to be more challenging, given the shear size of the dataset and necessity to extract the data trend. Our dataset consists of the daily values of 52 assets over 8 years. Among these assets were government bonds, variable income securities, fixed income securities, etc.

The bare return for each asset is given by:

$$\mu_{n,t}^{bare} \equiv \frac{P_{n,t} - P_{n,t-1}}{P_{n,t-1}}, \tag{28}$$

with $P_{n,t}$ the price at time $t$ of asset $n$. Importantly, mathematical expressions often call for the logarithmic returns instead of the bare returns. These are defined as

$$\mu_{n,t} \equiv \log\left(\frac{P_{n,t}}{P_{n,t-1}}\right) = \log\left(1 + \mu_{n,t}^{bare}\right). \tag{29}$$

There are several reasons why logarithmic returns are preferred over the bare returns [25]. In particular they follow a normal distribution, but most importantly for us, *they are time-additive*, and hence justify the sum in

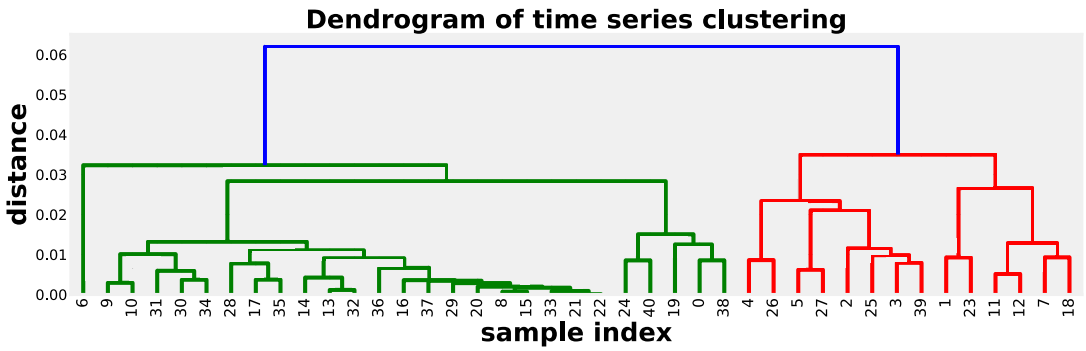


FIG. 3. Dendogram showing possible data clusters. The Euclidean distance between the different time series is shown on the vertical axis.

Eq.(1). In trading situations, though, the bare returns are usually small ($\mu_{n,t}^{bare} \ll 1$), implying that bare and logarithmic returns are interchangeable at the expense of a very small error, since

$$\mu_{n,t} = \log\left(1 + \mu_{n,t}^{bare}\right) \approx \mu_{n,t}^{bare} + \Theta\left((\mu_{n,t}^{bare})^2\right). \quad (30)$$

We used a dimensional reduction technique prior to applying our optimization routine. The motivation for this is two-fold.

First, most algorithms cannot tackle the problem in its full complexity. This is mainly true for the VQE algorithm, where dimensional reduction methods are key to solving the problem despite the processor's limited resources. This contrasts with the tensor networks and D-Wave Hybrid algorithms which can actually find close to optimal trajectories even when all of the problem's variables are used.

Second, we found that even the highest performance optimization algorithms tend to get stuck in local minima when the number of variables is truly gigantic. Dimensional reduction methods such as clustering serve the purpose of discarding irrelevant degrees of freedom early.

We have observed that assets which simultaneously present low variance and low returns tend to not be part of the optimal portfolio. Indeed, investing in these assets promises consistently low returns. By discarding assets which simultaneously have sub-average variance and sub-average returns, our dataset can be reduced from 52 down to 28 relevant assets.

Prior to clustering, we may also eliminate the noise in the data by applying a Hodrick-Prescott smoothing, which extracts the data trend. We then compute the Euclidean distance in the data trend for each pair of assets. This allows us to identify the degree of correlation between assets, which is represented as a dendogram in the diagram of Fig. 3. To select the optimal number of clusters we wish to consider, we plot the mean cluster variance versus the number of clusters (Fig. 4). We can clearly see that, beyond 8 clusters, increasing the number of clusters no longer significantly reduces the mean cluster variance. We therefore take $\approx 6 - 8$ as a reasonable choice of number of clusters for this dataset. The

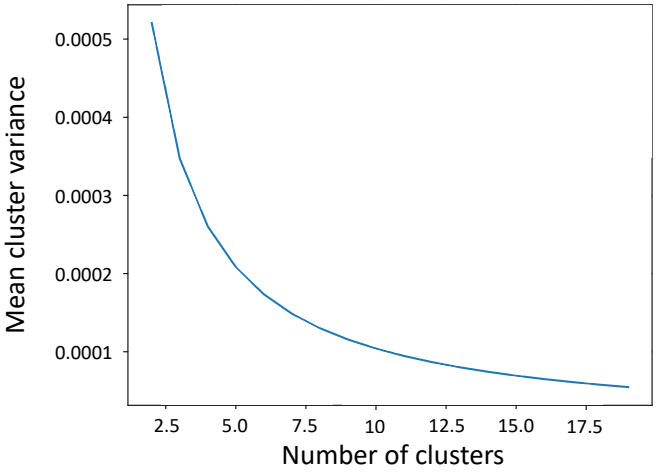


FIG. 4. Mean variance in each cluster versus the number of clusters.

assets' trend is shown in Fig. 5, grouped by cluster, for the case of 6 clusters. We observe a good agreement in general for the assets within each cluster. Only one of the clusters shows a relatively high variance between assets' trends. We could always address this issue by considering a larger number of clusters. Alternatively, we could also run several optimization rounds with variable numbers of clusters. By analogy to physics, we actually *renormalized* the assets. Within this picture, the dendrogram in Fig. 3 is nothing but the coarse-graining structure. In practice, after the clustering, the optimization is run over a cost function of *clusterized* assets, i.e., at every time step, the mean investment in each cluster $n$ is the component $\omega_n$ of the investment vector. In this work, we choose to equally distribute the total investment in a cluster among all the assets within the cluster, i.e., investing 10 on a cluster with 5 assets means for us an investment of 10/5 on each asset belonging to the cluster. Let us stress also that we only applied this clustering for specific methods and hardware, namely, for the ones that could not handle the full problem being analyzed (essentially VQE and VQE Constrained, which run on limited hardware).

We will consider the portfolio rebalancing to happen once monthly, for the sake of concreteness. The returns between transactions are calculated as the sum of returns during the course of that month. Note that we could have calculated them on the basis of a rolling average. This has the advantage of eliminating daily fluctuations, which make it easier to extract trends. On the flip side, the asset returns used in comparison would only approximately reflect the actual asset returns.

The daily covariance is an estimate of the daily fluctuations of a group of assets, encoded in the matrix $\Sigma_t$ at time $t$. We estimate the covariance at time $t$ based on the fluctuations in assets' returns over a window of the prior 20 business days. As a remark, notice also that clustered assets have, by construction, low covariance (since similar assets - i.e. those with large covariance - are the ones being clustered!). In practice, this also means having a smaller risk factor in the final cost function.

Once we get an optimal portfolio trajectory for clustered effective assets, we *unfold* the investment by assuming an equal investent on each asset forming each specific cluster. From this we obtain the daily return on an equal investment in each asset within each cluster, and in turn an overall trading trajectory in the original variables.

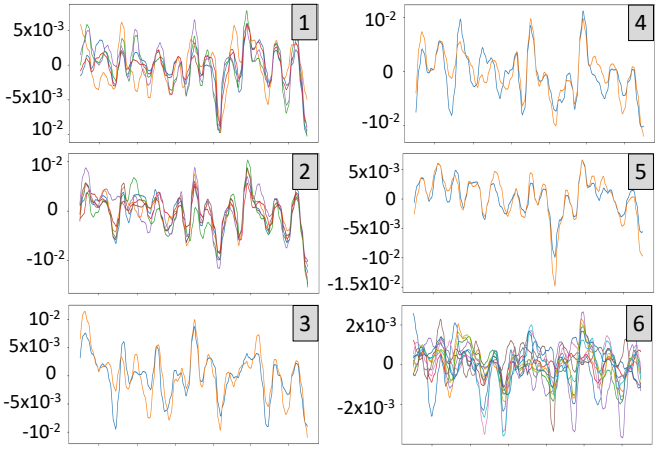


FIG. 5. Effective assets after clustering, from 52 down to 6. Horizontal axis is the period of time considered for this clustering (two years).

## V. RESULTS

Let us now discuss the results obtained with the different techniques [26] To show the capabilities of each solver, we built datasets of different sizes. We considered datasets of size XS, S, M, L, XL and XXL, each of which pushes a solver to its limit. The S dataset, for instance, is the maximal system size which the exhaustive solver could handle. Details of these datasets are shown in Table I. The risk aversion $\gamma$ represents a compromise between risk and returns. By tuning this parameter, we can find the portfolio which promises the highest returns for any fixed risk. The set of most profitable portfolios for all risks is known as the *efficient frontier*. In all simulations, and for the sake of simplicity, we set the risk aversion parameter $\gamma = 1$.

In Table II we show the comparison of Sharpe ratios obtained with the different methods and datasets, in Table III we show the profits (i.e., returns minus transaction costs, percentual), and in Table IV we show an estimation of the computational running time of our simulations. Entries were left blank when a dataset's size exceed a solver's capabilities.

Note that the solution which minimizes the cost function $\mathcal{H}$ does not always extremize the profit, returns, and Sharpe ratio. For dataset M for instance, the TN simulation produces a trajectory which presents higher profits and Sharpe ratio than other solvers, but is further away from the global minima. Another example is dataset L, for which Gekko produces the largest profits, but the largest Sharpe ratio is obtained with TNs instead (see Tables II and III). In fact, the performance of Gekko is quite remarkable, sometimes even better than quantum and quantum-inspired solutions depending on the metric, but unfortunately the method hits a memory wall around 500 qubits. In this sense, it would be nice to test also the performance of other quantum-inspired solutions such as Fujitsu's Digital Annealing, Microsoft QIO, and Amazon's Alpha-QUBO, as well as popular methods nowadays such as metaheuristics. Some of these techniques provide also a very good performance that would be worth analyzing. We leave this for future studies.

Rosenberg et al. were able so study a system of *24 fully connected qubits at most* (see Ref. [11] Table IV) using the capabilities of D-Wave's processor from 2016. By way of comparison, with today's D-Wave Hybrid as well as with our TN algorithm we optimized up to 1272 fully-connected qubits, and didn't hit the limit of the two approaches. From our experience, these two algorithms could well handle the full dataset of 28 relevant assets without any clustering and for reasonable periods of time. The algorithms are under continuous development, but this already shows important progress in the performance and variety of portfolio optimization strategies.

Concerning universal quantum processors, we observed good convergence of VQE for this problem in small datasets. As can be seen from Tables II, III and IV, a naive application of universal quantum processors such as IBM-Q still struggle to tackle optimization problems of commercially relevant dimensions. We were able to obtain good performance using appropriate data preparation. Note, however, that universal quantum processors have seen a phenomenal improvement over the past decade, such that we look forward to seeing progress in this area in the coming years.

We have observed that the D-Wave Hybrid approach works well for this problem, better in fact than a simple D-Wave approach (results not shown). Specifically, D-Wave Hybrid allows us to solve problems up to 1272 fully-connected qubits in 171 seconds, which in our opinion is

| Param. | XS | S | M | L | XL | XXL |
|---|---|---|---|---|---|---|
| $N$ | 3 | 4 | 4 | 8 | 8 | 8 |
| $N_t$ | 2 | 5 | 7 | 17 | 29 | 53 |
| $N_q$ | 1 | 1 | 1 | 2 | 2 | 3 |
| $N_{tot}$ | 6 | 20 | 28 | 272 | 464 | 1272 |
| $2^{N_{tot}}$ | 64 | $O(10^6)$ | $O(10^8)$ | $O(10^{81})$ | $O(10^{139})$ | $O(10^{382})$ |
| $K$ | 2 | 3 | 3 | 5 | 10 | 15 |
| $K'$ | 1 | 1 | 1 | 3 | 3 | 7 |

TABLE I. Specifics of the different datasets used for benchmarking the different algorithms and hardware platforms. Risk aversion for all datasets is fixed to $\gamma = 1$ and transaction costs to $\lambda = 1$. Time steps $N_t$ are measured in business months (i.e., not considering weekends due to closure of some markets).

| Method | XS | S | M | L | XL | XXL |
|---|---|---|---|---|---|---|
| VQE | 3.59 | - | - | - | - | - |
| Exhaustive | 6.31 | 8.90 | - | - | - | - |
| VQE Constrained | 6.31 | 6.04 | 4.81 | - | - | - |
| Gekko | 5.98 | 8.90 | 8.39 | 15.83 | 20.76 | - |
| D-Wave Hybrid | 5.98 | 8.90 | 8.39 | 7.47 | 9.70 | 12.16 |
| Tensor Networks | 5.98 | 8.90 | 9.54 | 16.36 | 15.77 | 15.83 |

TABLE II. Sharpe ratios computed by the different methods for the different datasets and time periods from Table I

*really* fast. We conclude that hybrid classical-quantum approaches can be already quite useful for quickly finding good-quality solutions to practical optimization problems. For the sake of comparison, the current version of our TN solver took 116833 seconds in solving the same problem running on a common laptop (a MacBook Pro using Matlab), and a pure quantum-annealing approach in D-Wave 2000Q would only have been able to solve up to 65 fully-connected qubits. We observe also that D-Wave hybrid provides an interesting landscape of potential solutions, see for instance Table V, where we show the energies as well as other relevant quantities computed by the algorithm for an optimization of 8 assets over 53 trading steps. As shown in that table, the different figures of merit are in fact different: minimizing the energy does not necessarily e.g. maximize profits.

We notice that our quantum-inspired TN solver tends to approach the problem's global minimum more reliably than D-Wave Hybrid in some cases. In the case of the M, L and XL datasets, for instance, our TN algorithm returns solutions which have a larger Sharpe ratio and/or larger profits. Furthermore, for the XXL dataset the solution could still be further improved by playing with different hyperparameters and fine-tuning the algorithm further, which is currently work in progress. In any case, it is interesting to notice that whereas D-Wave Hybrid provides the largest profits for the XXL dataset, the largest Sharpe ratio is however given by the TN solver. This deserves a deeper analysis, which we leave for future works.

| Method | XS | S | M | L | XL | XXL |
|---|---|---|---|---|---|---|
| VQE | 2.4 % | - | - | - | - | - |
| Exhaustive | 5.1 % | 13.9 % | - | - | - | - |
| VQE Constrained | 5.1 % | 9.1 % | 7.1 % | - | - | - |
| Gekko | 5.8 % | 13.9 % | 13.6 % | 54.1 % | 71.6 % | - |
| D-Wave Hybrid | 5.8 % | 13.9 % | 13.6 % | 18.9 % | 29.3 % | 67.6 % |
| Tensor Networks | 5.8 % | 13.9 % | 15.4 % | 38.2 % | 39.6 % | 39.7 % |

TABLE III. Profits (percentual) computed by the different methods for the different datasets and time periods from Table I.

| Method | XS | S | M | L | XL | XXL |
|---|---|---|---|---|---|---|
| VQE | 278 | - | - | - | - | - |
| Exhaustive | 0.005 | 34 | - | - | - | - |
| VQE Constrained | 123 | 412 | 490 | - | - | - |
| Gekko | 24 | 27 | 21 | 221 | 261 | - |
| D-Wave Hybrid | 8 | 39 | 19 | 52 | 74 | 171 |
| Tensor Networks | 0.838 | 51 | 120 | 26649 | 82698 | 116833 |

TABLE IV. Run-times (in seconds) estimated for the different methods for the different datasets from Table I.

We anticipate that our TN results can still be largely improved, both in terms of accuracy and performance. Based on our own estimations, we can reduce at least $\approx 100\times$ the run-time of our TN algorithm (see Sec. VI), as, in its present version, the TN algorithm is not memory intensive. In fact, acceleration of the TN code is possible at three levels: (i) software, (ii) hardware, and (iii) the algorithm itself. All in all, our work proves that TN solvers are a *cheap and practical* way to tackle large combinatorial optimization problems, compared to other quantum and quantum-inspired approaches.

## VI. NEXT STEPS

Let us now discuss what would be the next steps to make our algorithms better tailored to real-life situations, as well as to improve their efficiency.

### A. More constraints

As we said in the introduction, the portfolio optimization problem is central to quantitative finance. We have discussed its dynamic version, where we search for an optimal trading trajectory given the price history of several assets. There exist many commercially relevant variations of this problem. In this section, we will discuss three: the market impact, exact transaction costs, and the $10 - 5 - 40$ rule.

| T | E | C | R | TC | P | SR |
|---|---|---|---|---|---|---|
| 1 | -0.673 | 0.755 | 10.566 | 6.514 | 4.052 | 12.161 |
| 2 | -0.661 | 0.731 | 19.347 | 6.026 | 4.321 | 12.101 |
| 3 | -0.656 | 0.736 | 10.248 | 5.662 | 4.586 | 11.939 |
| 4 | -0.621 | 0.741 | 9.841 | 7.366 | 2.475 | 11.428 |
| 5 | -0.620 | 0.793 | 9.736 | 6.066 | 3.670 | 10.931 |

TABLE V. Example of landscape of computed solutions with D-Wave Hybrid, for an optimization of 8 assets over 53 trading steps, with maximum holdings 15: Computed Trajectory (T), Energy (E), Covariance (C), Returns (R), Transaction Costs (TC), Profit (P), and Sharpe Ratio (SR). The actual trajectory is not given for space reasons, but is available upon request.

### 1. *Market impact*

Intuitively, when a large order is passed, it can affect the market. For instance, a large "buy" order may increase the price of an asset because it signals high demand, whereas a large "sell" order may instead decrease it. This, of course, would alter the optimal trading trajectory. As discussed in Ref.[11] this can be implemented as

$$\text{Market} = \sum_{t=t_i}^{t_f} \Delta\overline{\omega}_t^T \Lambda_t' \overline{\omega}_t, \tag{31}$$

with $\Lambda_t$ a diagonal matrix of market impact coefficients.

Eq. (31) can be interpreted as follows: the impact of a trade in asset $n$ at time $t$ on the value of our portfolio is proportional to $\Delta\overline{\omega}_{n,t}$, the amount of shares of asset $n$ bought or sold, and to $\overline{\omega}_{n,t}$, the amount of shares of asset $n$ held at time $t$.

Note that if we are buying asset $n$, $\Delta\overline{\omega}_{n,t} > 0$, we expect the asset's value to increase. Our portfolio's returns should then grow as $\overline{\omega}_{n,t}$. The matrix $\Lambda_t'$ is therefore positive definite. Similarly, when selling, $\Delta\overline{\omega}_{n,t} < 0$, and our portfolio's returns drop with $\overline{\omega}_{n,t}$.

The resulting classical Hamiltonian including market impact is therefore

$$\begin{aligned}\mathcal{H} = \sum_{t=t_i}^{t_f} &- \mu_t^T \omega_t + \frac{\gamma}{2}\omega_t^T \Sigma_t \omega_t + \lambda(\Delta\omega_t)^2 \\ &- \Delta\omega_t^T \Lambda_t' \omega_t + \rho\left(u^T\omega_t - 1\right)^2,\end{aligned} \tag{32}$$

again in terms of normalized weigths. Note that this Hamiltonian is in the form of a QUBO.

As can be inferred from Eq. (31), this term is non-negligible only when (i) we have a large amount of the asset being traded in our portfolio, and (ii) the traded amount is large.

### 2. *Exact linear transaction costs*

As discussed in Sec. (II), we would expect the transaction costs to be linear in the absolute value of the transaction magnitude. We approximated these by a parabola, as polynomial expressions find a more natural formualtion as a QUBO. That option is cheap, fast, and works quite well for the accuracies that are normally considered. It is however possible to *exactly* implement the linear transaction costs by introducing $N \times N_t$ extra ancillary qubits $y_{n,t}$. This can be done by noticing that the sign of the difference $(\omega_{n,t+1} - \omega_{n,t})$ in Eq. (11) can be controlled by $y_{n,t}$ as follows:

$$\nu_{nt}|\omega_{n,t+1} - \omega_{n,t}| \to \nu_{nt}(\omega_{n,t+1} - \omega_{n,t})(1 - 2y_{n,t}), \tag{33}$$

with $y_{n,t} = 0$ if $(\omega_{n,t+1} - \omega_{n,t}) > 0$ and $y_{n,t} = 1$ otherwise. This condition can be further imposed by including the penalty term

$$\rho' \sum_{t=t_i}^{t_f} \sum_{n=1}^{N} (\omega_{n,t+1} - \omega_{n,t}) y_{n,t}, \tag{34}$$

with $\rho'$ a Lagrange multiplier. So, in this way, one could introduce the linear transaction costs in the QUBO formulation (notice that the above equations are quadratic), but at the expense of introducing more qubits, thus increasing the complexity of the optimization problem.

### 3. $10 - 5 - 40$ *rule*

It is common for investors to be bound by a constraint known as the $10 - 5 - 40$ rule: no investment in an asset can represent more than 10% of the total portfolio's value; investments larger than 5% of the portfolio's value cannot total 40% of the portfolio's value. In the following we will demonstrate a way to implement this constraint as a QUBO, but at the price of increasing the complexity of the optimization problem.

If we choose to impose the $10 - 5 - 40$ rule, we could proceed as follows. First, choose $K' = 0.1 \times K$ so that variables $\omega_{t,n} \in [0, K'/K]$ directly satisfy the 10% constraint, which can be achieved by choosing an appropriate $N_q$. Notice that, in our previous simulations, the ratios $K'/K$ can be extracted from Table I, but we could easily impose $K'/K = 0.1$. Next, we impose that the sum of those weights larger than 5% does not amount to more than 40% of the total investment. We can do this by adding a penalty term which uses *Slack variables* $\alpha_t \in \mathbb{R}$. The idea is that, by minimizing also over the Slack variables, one is able to convert an inequality constraint into an equality constraint. Additionally, we include ancillary qubits $y_{n,t}$ such that $y_{n,t} = 1$ if $\omega_{n,t} > 0.05$ and $y_{n,t} = 0$ otherwise. The penalty term for the 40% constraint is given by

$$\text{UpToForty} = \rho' \sum_{t=t_i}^{t_f} \left( \sum_{n=1}^{N} y_{n,t}\omega_{n,t} - 0.4 + \alpha_t^2 \right)^2, \tag{35}$$

with $\rho'$ a Lagrange multiplier. Finally, we need to ensure that $y_{n,t} = 1$ if and only if $\omega_{n,t} > 0.05$, which can be accounted for by the penalty term

$$\text{WhoIsFive} = \rho'' \sum_{t=t_i}^{t_f} \sum_{n=1}^{N} y_{n,t}(0.05 - \omega_{n,t} + \mu_{n,t}^2)^2 + (1 - y_{n,t})(\omega_{n,t} - 0.05 + \nu_{n,t}^2)^2, \quad (36)$$

which again uses a Lagrange multiplier $\rho''$ and Slack variables $\mu_{n,t}, \nu_{n,t} \in \mathbb{R}$.

If we include the $10-5-40$ rule, the optimization problem becomes much more complex because of two reasons: first, Eqs. (35) and (36) make this a higher-order optimization problem, since Eq (35) is quartic and Eq. (36) is cubic in the bit variables. Therefore, the problem is now a HUBO ("H" for "higher-order") instead of a QUBO. HUBOs are generally more complex than QUBOs, and there exist few computational architectures adapted for solving them (though they do exist). Second, the $10-5-40$ rule is a set of inequality constraints, which involve the inclusion of new hyperparameter Slack variables $\alpha_t, \mu_{n,t}$ and $\nu_{n,t}$ which also need to be optimized, thus increasing the complexity of the problem.

### B. Improved hardware and codes

We believe that it should be possible to handle bigger problems in a number of ways. Concerning quantum processors, D-Wave's processor"Advantage" with *Pegasus* topology is able to cope with a larger number of variables (5000+ qubits, 35000+ couplers, 15 couplers per qubits, 23x more optimal solutions than 2000Q for satisfiability problems and 20x faster), see for instance the numerical benchmarks in D-Wave's documentation [27]. Also, hybrid solutions such as D-Wave Hybrid benefit a lot from such developments. Concerning IBM-Q, we have found that a naive application of VQE was very rather limited for this problem, but obtained promising results by constraining the solution space using VQE. We think that it is worth exploring more hybrid solutions of this type. As a matter of fact, this is also the natural state of affairs: in the NISQ era, hybrid quantum-classical solutions (both in hardware and software) are offering the best operational performances, which is also somehow expected.

Our TNs code is able to handle $O(10^3)$ qubit variables using a MacBook Pro without problems, even though the performance can be improved in a number of ways. We believe that a highly optimized code in C++ fully parallelized on a HPC cluster should be able to handle *really* large problems very efficiently. GPU computing, FPGAs, and even Optical Processing Units (OPUs), should also provide improvements at the hardware level. Moreover, at the algorithmic level, it should be possible to try different TN strategies that should also improve the performance. The combination of all these easily mean more than 100× acceleration of the calculations. Since memory is not a constraint for our algorithm, this would also mean the ability to simulate much bigger systems than the ones considered in this paper. And of course, this option would still be cheaper than other alternatives.

## VII. CONCLUSION AND OUTLOOK

In this paper we have solved the problem of dynamic portfolio optimization using a number of quantum and quantum-inspired algorithms on different hardware platforms. We ran our solver on the daily values of 52 assets over 8 years. We implemented classical solvers (Gekko, exhaustive), D-Wave Hybrid quantum annealing, two different approaches based on VQE on IBM-Q (one of them brand-new, which we dubbed "VQE-Constrained"), and for the first time in this context also a quantum-inspired optimizer based on Tensor Networks. We also implemented a preprocessing based on clustering of assets. From our comparison, we conclude that D-Wave Hybrid and Tensor Networks are able to handle the largest possible systems in the present implementation, i.e., those for the XXL dataset. More specifically, with these two methods we managed to solve optimization problems up to 1272 fully-connected qubits, for demonstrative purposes and without hitting the limiting computational capabilities (we could in fact have targeted larger systems). For comparison, previous quantum dynamic portfolio optimizations involved up to 72 qubits (see Table VI in Ref.[11]). We observed also that D-Wave Hybrid is remarkably fast, whereas Tensor Networks sometimes provide better portfolios at the expense of a longer calculation time. To the best of our knowledge, our work is the first application of VQE and TNs to solve a dynamic portfolio optimization problem with real data. Moreover, our VQE-Constrained approach is also unique, as far as we know. Our preprocessing of real data in order to fit the current capabilities of quantum processors is also novel in this context.

From our results we also conclude that there seems to be no clear answer as to which is the "best" algorithm and hardware platform to solve the dynamic portfolio optimization problem for large systems. This is because there are several figures of merit at play: profits, Sharpe ratio, time cost, and also money cost. The performance of the algorithms is different depending on the figure of merit, leading us to conclude that, in practice, *the more options we have, the better.*

We have good reasons to think that the results presented in this paper are very promising. They show how real data can be handled by upcoming quantum computers, and also show the potential of quantum-inspired methods such as Tensor Networks. We also realize the importance of hybrid approaches, combining quantum and classical processing. This has been the key to im-

proved results, e.g., in VQE-Constrained and D-Wave Hybrid. In fact, a hybrid approach combining quantum processing and Tensor Networks should be quite successful for many problems. This is a topic that we are currently investigating in the broad sense. We also believe that all these developments, involving quantum and quantum-inspired techniques, will change the way quantitative finance is done, and for good.

**Acknowledgments.-** We thank Hossein Sadeghi for insight about D-Wave, Mhedi Bozzo-Rey for insight about IBM-Q, as well as Christophe Jurczak, Pedro Luis Uriarte, Pedro Muñoz-Baroja, Creative Destruction Lab, BIC-Gipuzkoa, DIPC, Ikerbasque, and Basque Government for constant support.

**Competing Interests.-** The authors declare that there are no competing interests.

**Data Availability.-** The datasets generated during and/or analysed during the current study are available from the corresponding author on a reasonable request.

**Code Availability.-** The code used during the current study is available from the corresponding author on a reasonable request.

**Author Contribution.-** All authors contributed to the generation of ideas, algorithm design, algorithm implementation, analysis of results, and writing of the paper.

[1] Harry Markowitz, "Portfolio selection," The Journal of Finance **7**, 77–91 (1952).

[2] Román Orús, Samuel Mugel, and Enrique Lizaso, "Quantum computing for finance: Overview and prospects," Reviews in Physics **4**, 100028 (2019).

[3] Daniel J. Egger, Claudio Gambella, Jakub Marecek, Scott McFaddin, Martin Mevissen, Rudy Raymond, Andrea Simonetto, Stefan Woerner, and Elena Yndurain, "Quantum computing for finance: state of the art and future prospects," (2020), arXiv:2006.14510.

[4] P. Rebentrost, B. Gupt, and T. R. Bromley, "Quantum computational finance: Monte carlo pricing of financial derivatives," (2018), arXiv:1805.00109.

[5] Nikitas Stamatopoulos, Daniel J. Egger, Yue Sun, Christa Zoufal, Raban Iten, Ning Shen, and Stefan Woerner, "Option pricing using quantum computers," ArXiv:1905.02666.

[6] Ana Martin, Bruno Candelas, Ángel Rodriguez-Rozas, José D. Martin-Guerrero, Xi Chen, Lucas Lamata, Román Orús, Enrique Solano, and Mikel Sanz, "Towards pricing financial derivatives with an ibm quantum computer," ArXiv:1904.05803.

[7] Román Orús, Samuel Mugel, and Enrique Lizaso, "Forecasting financial crashes with quantum computing," Phys. Rev. A **99**, 060301 (2019).

[8] Yongcheng Ding, Lucas Lamata, José D. Martin-Guerrero, Enrique Lizaso, Samuel Mugel, Xi Chen, Román Orús, Enrique Solano, and Mikel Sanz, "Towards prediction of financial crashes with a d-wave quantum computer," (2019), arXiv:1904.05808.

[9] G. Rosenberg, "Finding optimal arbitrage opportunities using a quantum annealer," (2016).

[10] A. Milne, M. Rounds, and P. Goddard, "Optimal feature selection in credit scoring and classification using a quantum annealer," (2017).

[11] Gili Rosenberg, Poya Haghnegahdar, Phil Goddard, Peter Carr, Kesheng Wu, and Marcos López De Prado, "Solving the Optimal Trading Trajectory Problem Using a Quantum Annealer," IEEE Journal of Selected Topics in Signal Processing **10**, 1053–1060 (2016).

[12] M. López de Prado, "Generalized optimal trading trajectories: A financial quantum computing application," (2015), 10.2139/ssrn.2575184, http://dx.doi.org/10.2139/ssrn.2575184.

[13] Logan D.R. Beal, Daniel C. Hill, R. Abraham Martin, and John D. Hedengren, "GEKKO optimization suite," Processes **6** (2018), 10.3390/pr6080106.

[14] An obvious but non-trivial remark: imposing $\sum_{n=1}^{N} \overline{\omega}_{n,t} \leq K$ instead, may actually produce portfolios with higher returns. Notice also that $K$ is the investment, and *not* the value of the investment, e.g. $K = 3$ apples, not the value (e.g. in dollars) of those 3 apples.

[15] A different splitting amounts to a rescaling of the optimal Lagrange multiplier $\rho$, without changing the actual solution.

[16] F Barahona, "On the computational complexity of ising spin glass models," Journal of Physics A: Mathematical and General **15**, 3241–3253 (1982).

[17] E. Farhi, J. Goldstone, S. Gutmann, and M. Sipser, "Quantum computation by adiabatic evolution," (2000), quant-ph/0001106.

[18] Alberto Peruzzo, Jarrod McClean, Peter Shadbolt, Man-Hong Yung, Xiao-Qi Zhou, Peter J. Love, Alán Aspuru-Guzik, and Jeremy L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," Nature Communications **5**, 4213 (2014).

[19] John Preskill, "Quantum Computing in the NISQ era and beyond," Quantum **2**, 79 (2018).

[20] Maria Schuld, Alex Bocharov, Krysta M. Svore, and Nathan Wiebe, "Circuit-centric quantum classifiers," Phys. Rev. A **101**, 032308 (2020).

[21] Ville Bergholm, Josh Izaac, Maria Schuld, Christian Gogolin, M. Sohaib Alam, Shahnawaz Ahmed, Juan Miguel Arrazola, Carsten Blank, Alain Delgado, Soran Jahangiri, Keri McKiernan, Johannes Jakob Meyer, Zeyue Niu, Antal Szava, and Nathan Killoran, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," (2018), arXiv:1811.04968.

[22] Román Orús, "A practical introduction to tensor networks: Matrix product states and projected entangled pair states," Annals of Physics **349**, 117 – 158 (2014).

[23] Román Orús, "Tensor networks for complex quantum systems," Nature Reviews Physics **1**, 538–550 (2019).

[24] M. Fannes, B. Nachtergaele, and R. F. Werner, "Finitely correlated states on quantum spin chains," Communications in Mathematical Physics **144**, 443–490 (1992).

[25] See, e.g., https://en.wikipedia.org/wiki/Rate_of_return.
[26] All portfolio optimizers have been developed by the Multiverse Computing© team.
[27] Catherine McGeoch and Pau Farré, "The d-wave advantage system: An overview," D-Wave Whitepaper (2020).